Master-Rolladensteuerung V88

Home Assistant Blueprint – Sonnenstandsbeschattung, Extrem-Hitzeschutz, Lüftungs- und Durchgangslogik

Vollständige Dokumentation · Stand August 2026 · Version V88 · Blueprint-Domain: automation

Inhalt

1. Was der Blueprint macht
2. Funktionsprinzip: die Prioritätenkette
3. Voraussetzungen und benötigte Entitäten
4. Helfer und Template-Sensoren anlegen
5. Fenstersensorik: Kippstellung vs. ganz offen
6. Installation des Blueprints
7. Vollständige Input-Referenz
8. Einstellempfehlungen nach Himmelsrichtung
9. Der manuelle Override
10. Zusammenspiel mit einer Brandalarm-Automatisierung (inkl. quittiertem Reset und Dashboard-Karte)
11. Fehlersuche und Debugging
12. Bekannte Einschränkungen
13. Changelog

1. Was der Blueprint macht

Der Blueprint steuert ein einzelnes Rollo vollautomatisch über den Tag. Für jedes Rollo im Haus wird eine eigene Automatisierung aus dem Blueprint erzeugt – so kann jeder Raum eigene Himmelsrichtungen, Temperaturschwellen und Positionen bekommen.

Er vereint sechs Aufgaben, die man sonst in einzelnen Automatisierungen bauen müsste:

- **Morgendliches Öffnen** – abhängig von Uhrzeit, Sonnenaufgang, Präsenz, Wochentag, Feiertag und Hausmodus.
- **Sonnenstandsbeschattung** – das Rollo fährt nur dann herunter, wenn die Sonne tatsächlich auf diese Fassade scheint (Azimut-Fenster je Himmelsrichtung), es hell genug ist (PV-Leistung oder Lux) und die Temperatur das rechtfertigt.
- **Extrem-Hitzeschutz** – bei sehr hoher Innen- und Außentemperatur fährt das Rollo auf eine separate, deutlich weiter geschlossene Position. Mit Hysterese gegen Pendeln.
- **Lüftungsposition** – gekipptes Fenster hält das Rollo auf einer definierten Mindesthöhe.
- **Durchgangs- und Querlüftungslogik** – ganz geöffnetes Fenster / Terrassentür fährt das Rollo hoch, damit man weder eingesperrt wird noch von Hand nachhelfen muss.
- **Abendliches Schließen** – nach Sonnenuntergang mit einstellbarem Versatz, unter Berücksichtigung offener Fenster.

> **Kernidee:** Der Blueprint entscheidet nie nach einer einzelnen Bedingung, sondern nach einer festen Prioritätenkette. Sicherheit schlägt Komfort, Komfort schlägt Energie.

2. Funktionsprinzip: die Prioritätenkette

Bei jedem Auslöser durchläuft der Blueprint eine `choose`-Kette von oben nach unten. Der erste zutreffende Zweig gewinnt, alle weiteren werden übersprungen.

Prio	Zweig	Bedingung und Wirkung
0	Brandalarm	Ist der Brandalarm-Boolean `on`, wird die Ausführung sofort mit `stop` beendet. Der Blueprint fährt nichts – er hält still, damit er eine externe Brandalarm-Automatisierung (die alle Rollos als Fluchtweg öffnet) nicht wieder zufährt. Siehe Kapitel 10.
1a	Durchgang / Querlüften	Ein Sensor aus *Fenster ganz offen* meldet `on`. Das Rollo fährt hoch: • ist keine Beschattung aktiv → auf `max_oeffnung` (ganz auf) • ist Beschattung oder Extrem-Hitzeschutz aktiv → nur auf `durchgangs_pos` Wirkt auch nachts, vor der Morgen-Öffnungszeit und aus manuell geschlossenem Zustand heraus. Das Rollo wird dabei nie zugefahren, nur hoch.
1b	Lüftung (Kipp)	Ein Sensor aus *Fenster Lüftung* meldet `on` und kein Ganz-offen-Sensor ist aktiv. Das Rollo fährt mindestens auf `lueftungs_pos`, damit der Rollopanzer nicht auf dem gekippten Flügel aufsetzt.
2	Abend-Check	Zur eingestellten Kontrollzeit (Default 22:00) prüft der Blueprint, ob ein Fenster noch ganz offen steht, und schickt eine Push-Nachricht.
3	Schließen	**Nacht (Sonnenuntergang plus `sunset_offset`)** und kein Fenster ganz offen → Rollo schließt. Bei gekipptem Fenster wird stattdessen nur bis auf `lueftungs_pos` geschlossen. **Der Versatz steckt seit V87 in der Nacht-Bedingung selbst und gilt damit für jeden Auslöser, nicht nur für den Sonnenuntergangs-Trigger.**
4a	Extrem-Hitzeschutz	Sonne auf der Fassade *und* Innentemperatur über der EIN-Schwelle *und* Außentemperatur über der EIN-Schwelle → Rollo fährt auf `hitze_extrem_pos`. Der Modus endet erst, wenn Innen- oder Außentemperatur unter die jeweilige AUS-Schwelle fällt (Hysterese).
4b	Normale Beschattung	Sonne auf der Fassade *und* Hitze-Alarm *und* genug Licht (PV oder Lux) → Rollo fährt auf die Beschattungsposition. Vor- und nachmittags getrennt einstellbar.
4c	Auffahren	Kein Sonnenschutz nötig (Sonne weg, bewölkt, kühl) → Rollo fährt auf `max_oeffnung`.

Wann läuft die Automatisierung überhaupt?

Auslöser sind: Zustandswechsel der Fenstersensoren, Präsenzmelder, PV-Leistung (mit 2 min Entprellung), Sonnenuntergang, die Zeitpunkte *spätestens öffnen* und *Abend-Check*, ein Wechsel des Hausmodus – sowie ein `time_pattern` **alle 5 Minuten** als Sicherheitsnetz. Der Modus ist `restart`: ein neuer Auslöser bricht einen laufenden Durchlauf ab.

Seit V87 gibt es keinen eigenen Trigger für den Sonnenuntergangs-Versatz. Das Sicherheitsnetz löst den Schließvorgang aus, sobald die Nacht-Bedingung kippt – die maximale Verzögerung beträgt damit 5 Minuten. Ein `delay` wäre an dieser Stelle wirkungslos, weil `mode: restart` jeden laufenden Durchlauf beim nächsten Auslöser abbricht.

Die drei Sonnenbedingungen im Detail

- `sonne_auf_f` – Der Sonnen-Azimut liegt im Winkelbereich der gewählten Himmelsrichtung und die Elevation ist über 5°. Verhindert Beschattung, wenn die Sonne gar nicht auf das Fenster scheint.
- `sonne_stark` – PV-Leistung über der Schwelle oder Lux über der Schwelle, jeweils plausibilisiert mit der Wetterentität. Fällt ein Sensor aus, greift ein Failover allein über das Wetter.
- `hitze_alarm` – Verknüpft ODER: Innentemperatur über der Innenschwelle, oder steigender Temperaturtrend, oder Außentemperatur über der Außenschwelle. Eine Winter-Sperre (`raumtemp_min_schwelle`) unterdrückt alles, solange es im Raum zu kühl ist.

Wichtig zu verstehen: Weil `hitze_alarm` eine ODER-Verknüpfung ist, macht eine sehr niedrig gesetzte Außenschwelle (z. B. 10 °C) die Innentemperatur praktisch wirkungslos – ab 10 °C draußen ist die Bedingung im Sommer immer wahr, und es entscheiden nur noch Sonnenstand und Helligkeit. Das kann gewollt sein (»die Sonne blendet mich auch bei 15 °C«), sollte aber eine bewusste Entscheidung sein.

3. Voraussetzungen und benötigte Entitäten

Was	Pflicht	Beschreibung
`cover.*`	ja	Rollo mit Positionsunterstützung (`set_cover_position` und Attribut `current_position`). Reine Auf/Zu-Rollos funktionieren nicht.
`sun.sun`	ja	Standard-Integration. Liefert Azimut und Elevation.
`weather.*`	ja	Beliebige Wetterentität. Dient zur Plausibilisierung und als Failover.
Raumtemperatursensor	ja	`sensor` mit `device_class: temperature` .
Außentemperatursensor	ja	Eigener Sensor oder z. B. OpenWeatherMap.
input_select Hausmodus	ja	Helfer mit mindestens den Optionen *Zuhause*, *Abwesend*, *Urlaub*.
input_boolean Brandalarm	ja	Wird von der externen Brandalarm-Automatisierung gesetzt.
Fenstersensor Kipp	nein	`binary_sensor` , Mehrfachauswahl möglich.
Fenstersensor ganz offen	nein	`binary_sensor` , Mehrfachauswahl möglich. Leer lassen bei Fenstern, die nie ganz geöffnet werden.
Präsenzmelder	nein	Mehrfachauswahl. Löst den morgendlichen »Startschuss« aus.
PV-Leistungssensor	nein	W oder kW – die Einheit wird automatisch erkannt und umgerechnet.
Helligkeitssensor	nein	`device_class: illuminance` . Pro Fassadenseite ein Sensor ist ideal.
Trend-Sensor	nein	`binary_sensor` , `on` = Raumtemperatur steigt. Siehe Kapitel 4.
Workday-Sensor	nein	Für Feiertagserkennung. Siehe Kapitel 4.
timer Benachrichtigungspause	nein	Unterdrückt Push-Nachrichten für eine gewisse Zeit.
Notify-Dienst	nein	Empfehlung: eine Notify-Gruppe, siehe Kapitel 4.

4. Helfer und Template-Sensoren anlegen

4.1 Hausmodus (input_select)

Einstellungen → Geräte & Dienste → Helfer → Helfer erstellen → Dropdown. Name: *Hausmodus*. Optionen exakt so schreiben – der Blueprint vergleicht Textwerte:

```
Zuhause
Abwesend
Urlaub
```

Wirkung: *Urlaub* schaltet auf die späte Öffnungszeit. *Abwesend* und *Urlaub* erlauben das Öffnen auch ohne Präsenzmeldung.

4.2 Brandalarm-Status (input_boolean)

Helfer erstellen → Umschalter. Name z. B. *Brandalarm Status* → Entität `input_boolean.brandalarm_status` . Dieser Boolean ist die Schnittstelle zwischen Rauchmelder-Automatisierung und Rolladensteuerung.

4.3 Benachrichtigungs-Pause (timer)

Helfer erstellen → Timer. Name z. B. *Rollo Pause für Benachrichtigung*. Solange dieser Timer läuft, gilt die Push-Benachrichtigung als unterdrückt.

4.4 Workday-Sensor (Feiertage)

Einstellungen → Geräte & Dienste → Integration hinzufügen → Workday. Land und Bundesland wählen, Wochenende als arbeitsfreie Tage markieren. Ergebnis: `binary_sensor.workday_sensor` , der an Wochenenden und Feiertagen `off` ist.

Der Blueprint nutzt ihn so: `off` → es gilt die späte Öffnungszeit (WE/Urlaub). Bleibt der Input leer, fällt der Blueprint auf eine reine Wochentagsprüfung zurück – Feiertage werden dann nicht erkannt.

4.5 Trend-Sensor für die Raumtemperatur

Der Trend-Sensor macht die Beschattung vorausschauend: Steigt die Raumtemperatur, wird beschattet, bevor die Schwelle erreicht ist. Anlegen über die Trend-Integration (Einstellungen → Geräte & Dienste → Integration hinzufügen → Trend) oder per YAML:

```yaml
binary_sensor:
  - platform: trend
    sensors:
      temp_wohnzimmer_rollo:
        friendly_name: "Temp Wohnzimmer steigt"
        entity_id: sensor.temperatur_wohnzimmer
        sample_duration: 1800     # 30 Minuten Beobachtungsfenster
        min_gradient: 0.0005      # ca. +0,9 K/h
        device_class: heat
```

Pro Raum einen solchen Sensor anlegen und in der jeweiligen Automatisierung eintragen.

4.6 Notify-Gruppe (empfohlen)

Statt in jeder Automatisierung den Gerätedienst einzutragen, sollte eine Notify-Gruppe verwendet werden. Beim Gerätewechsel ändert man dann nur eine Zeile. In der `configuration.yaml` :

```
notify:
  - platform: group
    name: handys
    services:
      - service: mobile_app_mein_handy
      - service: mobile_app_handy_partner   # optional
```

Anschließend Home Assistant neu starten (ein Reload reicht für Notify-Plattformen nicht). Im Blueprint wird dann als *Handy Benachrichtigungs-Dienst* nur `handys` eingetragen – der Blueprint baut daraus `notify.handys`.

> Notify-Gruppen sind Legacy-Dienste und erzeugen keine Entität. Sie tauchen daher nicht unter »Zustände« auf, sondern nur unter Entwicklerwerkzeuge → Aktionen.

5. Fenstersensorik: Kippstellung vs. ganz offen

Der Blueprint unterscheidet zwei Fensterzustände, weil sie unterschiedliche Konsequenzen haben:

- **Gekippt** → Rollo darf herunter, aber nur bis zur Lüftungsposition (sonst setzt der Panzer auf dem Flügel auf).
- **Ganz offen** → Rollo muss hoch (Durchgang, Querlüften, kein Einsperren).

Ein einzelner Reed-Kontakt kann das nicht unterscheiden – er sieht nur »Flügel bewegt sich vom Rahmen weg«. Es gibt drei praktikable Wege:

Variante A – Zwei Reed-Kontakte (empfohlen, am robustesten)

Ein Kontakt sitzt so, dass er beim Kippen öffnet, ein zweiter so, dass er erst beim vollständigen Öffnen öffnet (typischerweise an der Schließseite des Flügels). Beide Sensoren werden direkt in die jeweiligen Blueprint-Inputs eingetragen – keine Templates nötig. Das ist die Variante mit dem geringsten Fehlerpotenzial.

Variante B – Griffsensor plus Fensterkontakt

Ein Fenstergriffsensor (z. B. Aqara, Bosch) liefert drei Zustände: geschlossen / gekippt / offen. In Kombination mit einem Öffnungskontakt lassen sich beide Zustände per Template-Helfer ableiten:

```
template:
  - binary_sensor:
      - name: "Fenster Büro Kippstellung"
        unique_id: fenster_buero_kippstellung
        device_class: window
        state: >
          {{ is_state('sensor.fenstergriff_buero', 'tilt') }}

      - name: "Fenster Büro ganz geöffnet"
        unique_id: fenster_buero_ganz_geoeffnet
        device_class: window
        state: >
          {{ is_state('sensor.fenstergriff_buero', 'open') }}
```

Die konkreten Zustandsnamen (`open`, `tilt`, `closed`) hängen vom Gerät und der Integration ab – bitte in den Entwicklerwerkzeugen unter »Zustände« nachsehen.

Variante C – Fensterkontakt plus separater Griffkontakt

Wenn nur ein Öffnungskontakt (`A`) und ein Kontakt am Griff (`B`) vorhanden sind, lässt sich die Logik so ableiten:

```
template:
  - binary_sensor:
      - name: "Fenster Küche Kippstellung"
        unique_id: fenster_kueche_kippstellung
        device_class: window
        state: >
          {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
             and is_state('binary_sensor.griff_kueche_contact', 'on') }}

      - name: "Fenster Küche ganz geöffnet"
        unique_id: fenster_kueche_ganz_geoeffnet
        device_class: window
        state: >
          {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
             and is_state('binary_sensor.griff_kueche_contact', 'off') }}
```

> **Fenster ohne Ganz-offen-Sensor:** Den Input *Fenster ganz offen* einfach leer lassen. Der Durchgangs-Zweig greift dann für dieses Rollo nie – alles andere funktioniert weiter. Sinnvoll bei Fenstern, die ohnehin nur gekippt werden (Gäste-WC, Kellerfenster).

> **Fallstrick:** Wird derselbe Kontakt sowohl als Kipp- als auch als Ganz-offen-Sensor eingetragen, blockiert die Ganz-offen-Logik die Lüftungslogik dauerhaft. Immer nur einen der beiden Inputs pro physischem Sensor belegen.

6. Installation des Blueprints

1. Die YAML-Datei ablegen unter: `config/blueprints/automation/<Ordnername>/<Dateiname>.yaml`
2. Entwicklerwerkzeuge → YAML → Automatisierungen neu laden. Bei Änderungen an der `configuration.yaml` stattdessen einmal komplett neu starten.
3. Einstellungen → Automatisierungen & Szenen → Blueprints → den Blueprint auswählen → Automatisierung erstellen.
4. Alle Inputs für das erste Rollo ausfüllen, speichern, benennen.
5. Für jedes weitere Rollo wiederholen.

Wird der Blueprint später aktualisiert, genügt es, die Datei zu ersetzen und die Automatisierungen neu zu laden. Die eingetragenen Inputs bleiben erhalten, solange die Input-Namen gleich bleiben. Entfernte Inputs werden ignoriert, neue Inputs greifen auf ihren Default zurück.

> Wird der Blueprint über eine URL importiert (Blueprint erneut importieren), zieht Home Assistant die Datei direkt von der Quelle. Eine lokal ersetzte Datei wird dabei überschrieben – beide Wege also nicht mischen.

7. Vollständige Input-Referenz

Grundlagen

Input	Default	Bedeutung
`rollo_entity`	–	Das zu steuernde Rollo (`cover`).
`himmelsrichtung`	–	Azimut-Fenster der Fassade. Bestimmt, wann »die Sonne auf dieses Fenster scheint«. Bei ungewöhnlichen Fassaden die Option wählen, deren Winkelbereich am besten passt.
`brandalarm_entity`	`input_boolean.brandalarm_status`	Solange `on`, führt der Blueprint keinerlei Fahrten aus.
`hausmodus_entity`	–	Der Hausmodus-Helfer.
`weather_entity`	–	Wetterentität zur Plausibilisierung.

Positionen (0 % = geschlossen, 100 % = offen)

Input	Default	Bedeutung
`max_oeffnung`	94	Normale Offen-Position. Bewusst unter 100 % wählen, wenn der Rollopanzer sonst mechanisch anschlägt oder die Position bei »ganz oben« nicht sauber zurückgemeldet wird.
`beschattung_pos_morgens`	60	Beschattungsposition vor der Wechselzeit.
`beschattung_pos_nachmittags`	60	Beschattungsposition nach der Wechselzeit.
`beschattung_wechsel_zeit`	13:00	Umschaltpunkt zwischen den beiden Beschattungspositionen. Richtwert: der Zeitpunkt, an dem die Sonne die Fassade voll trifft.
`lueftungs_pos`	15	Mindesthöhe bei gekipptem Fenster. **Ausdrücklich eine Untergrenze, keine Fixposition: Steht das Rollo höher, bleibt es stehen.**
`durchgangs_pos`	80	Position bei ganz geöffnetem Fenster, während Beschattung oder Extrem-Hitzeschutz aktiv sind. Ohne aktiven Sonnenschutz fährt das Rollo stattdessen auf `max_oeffnung`.
`hitze_extrem_pos`	10	Position bei extremer Hitze.

Temperaturschwellen

Input	Default	Bedeutung
`raumtemp_max_schwelle`	23,5 °C	Ab dieser Innentemperatur gilt Hitze-Alarm. Bei bereits aktiver Beschattung rechnet der Blueprint intern mit Schwelle minus 1 K als Hysterese.
`raumtemp_min_schwelle`	21,0 °C	Winter-Sperre. Unterhalb dieser Innentemperatur wird nie beschattet – die Sonne darf heizen.
`aussentemp_schwelle`	25,0 °C	Außentemperatur, ab der beschattet werden darf. ODER-verknüpft mit der Innenschwelle.
`hitze_extrem_innen_ein`	25,0 °C	Innenschwelle, ab der der Extrem-Modus einsetzt.
`hitze_extrem_innen_aus`	23,5 °C	Innenschwelle, unter der der Extrem-Modus endet.
`hitze_extrem_aussen_ein`	28,0 °C	Außenschwelle für den Extrem-Modus.
`hitze_extrem_aussen_aus`	26,0 °C	Außenschwelle, unter der der Extrem-Modus endet.

> **Mindestabstand einhalten:** Zwischen `raumtemp_min_schwelle` und `raumtemp_max_schwelle` sollten mindestens 2 K liegen. Bei zu engem Fenster (z. B. 21 / 22 °C) fallen Ein- und Ausschaltpunkt praktisch zusammen und das Rollo pendelt. Bewährt: 20 °C / 22,5 °C.

Helligkeit und PV

Input	Default	Bedeutung
`pv_sensor`	leer	PV-Leistung. W oder kW – wird automatisch erkannt.
`pv_schwelle`	2000 W	Ab dieser Leistung gilt es als »hell«. Beim Doppelten der Schwelle (»es knallt«) wird die Wetterprüfung übersprungen.
`lux_sensor`	leer	Helligkeitssensor, idealerweise pro Fassadenseite.
`lux_schwelle`	25000	Ab diesem Wert gilt es als »hell«. Der Default ist für Außensensoren im Schatten oft zu hoch – bei einem Sensor an der Fassade liegen praxistaugliche Werte eher bei 9.000–15.000 lx.

Fallen beide Sensoren aus oder sind sie leer, entscheidet allein die Wetterentität (Failover).

Zeiten und Präsenz

Input	Default	Bedeutung
`zeit_fruehestens`	06:30	Frühestes Öffnen an Werktagen.
`sonnenaufgang_woche`	an	Wenn aktiv, wird an Werktagen zusätzlich auf den Sonnenaufgang gewartet.
`zeit_we_fruehestens`	08:00	Frühestes Öffnen an Wochenenden, Feiertagen und im Urlaubsmodus.
`zeit_spaetestens`	09:30	Harter Fallback: zu dieser Zeit wird geöffnet, auch ohne Präsenz.
`zeit_oeffnen_sperre`	11:00	Ende der Morgenphase. Danach wird ein geschlossenes Rollo als manuell geschlossen gewertet (siehe Kapitel 9).
`zeit_kontrolle`	22:00	Zeitpunkt des Abend-Checks auf offene Fenster.

`sunset_offset`	30 min	**Verzögerung nach Sonnenuntergang bis zum Beginn des Nachtmodus.** Negative Werte = früher schließen. **Seit V87 wirkt der Wert auf die Nacht-Bedingung selbst und damit auf jeden Auslöser (korrekt umgesetzt erst ab V88).**
`praesenz_sensor`	leer	Ein oder mehrere Präsenzmelder. Meldet einer `on` und die früheste Zeit ist erreicht, wird geöffnet.
`workday_sensor`	leer	Feiertagserkennung. `off` = späte Öffnungszeit.

> **Genauigkeit des Sonnenuntergangs-Versatzes:** Der heutige Sonnenuntergang wird aus `next_setting` minus einem Tag abgeleitet. Um die Tagundnachtgleichen verschiebt sich der Sonnenuntergang um bis zu ca. 6 Minuten pro Tag, der berechnete Nachtbeginn liegt dann entsprechend daneben – im Herbst etwas zu früh, im Frühjahr etwas zu spät. Im Hochsommer und Winter geht die Abweichung gegen null. Bei Versätzen von 30–60 Minuten ohne praktische Bedeutung, bei sehr kurzen Werten gut zu wissen.

Benachrichtigungen

Input	Default	Bedeutung
`notify_handy`	leer	Der Dienstname ohne Präfix. Bei `handys` ruft der Blueprint `notify.handys` auf.
`notify_block_active`	aus	Blockade-Benachrichtigungen aktivieren.
`pause_timer`	leer	Timer, der Benachrichtigungen unterdrückt, solange er läuft.

8. Einstellempfehlungen nach Himmelsrichtung

Fassade	Wechselzeit	Beschattung vm / nm	Begründung
Ost	ca. 10:00	20–30 % / 30 %	Frühe, flache Sonne blendet stark. Nach Mittag ist die Fassade im Schatten – die Beschattung endet dann automatisch, weil `sonne_auf_f` falsch wird.
Süd	ca. 11:00–13:00	50–65 % / 30–65 %	Hochstehende Sonne. Oft reicht eine teilweise Beschattung, weil der Dachüberstand mithilft.
West	ca. 15:00	60 % / 40 %	Nachmittags flache, aggressive Sonne. Nachmittags weiter schließen als vormittags – die häufigste Fehlkonfiguration ist genau die umgekehrte Reihenfolge.
Nord	beliebig	–	Beschattung meist überflüssig. Blueprint trotzdem sinnvoll für Lüftung, Nachtabsenkung und Durchgang.

Vorgehen bei der Erstinbetriebnahme

1. Erst ein Rollo einrichten und ein bis zwei sonnige Tage beobachten.
2. Fährt es zu selten herunter? → Lux-Schwelle oder PV-Schwelle senken.
3. Fährt es zu oft oder bei bewölktem Himmel herunter? → Schwellen anheben oder die Wetterprüfung wirken lassen.
4. Pendelt es? → Abstand zwischen `raumtemp_min` und `raumtemp_max` vergrößern.
5. Erst wenn ein Rollo sauber läuft, die Einstellungen auf die übrigen übertragen.

9. Der manuelle Override

Der Blueprint versucht zu erkennen, ob ein Rollo von Hand auf eine Position gefahren wurde, die er selbst nie anfahren würde. Ist das der Fall, hält er sich mit der Tag-Logik zurück und respektiert die Entscheidung des Nutzers.

Die Erkennung funktioniert über die aktuelle Position. Als »automatisch« gelten – mit ±2 % Toleranz:

- `max_oeffnung`, beide Beschattungspositionen, `lueftungs_pos`, `durchgangs_pos`, `hitze_extrem_pos`
- vollständig geschlossen (≤ 5 %)

Jede andere Position gilt als manueller Override. Zusätzlich gilt: ein geschlossenes Rollo nach der Morgen-Öffnungssperre wird ebenfalls als manuell geschlossen gewertet – wer morgens um 11 die Rollos wieder herunterlässt, will offenbar Ruhe haben.

> **Wichtig:** Der Override bremst nur die Tag-Logik (Beschattung, Auffahren, Extrem-Hitzeschutz). Er wird bewusst ignoriert bei: Brandalarm, Durchgang/Querlüften und dem abendlichen Schließen. Sicherheit und Fensterlogik haben immer Vorrang.

> **Nebenwirkung beim Einstellen:** Wenn zwei Positionen zufällig sehr nah beieinander liegen (z. B. Lüftung 30 % und Beschattung 30 %), kann der Blueprint sie nicht mehr unterscheiden. Positionen mit mindestens 10 % Abstand wählen.

10. Zusammenspiel mit einer Brandalarm-Automatisierung

Der Blueprint öffnet bei Brandalarm nicht selbst – er hält still. Das Öffnen als Fluchtweg übernimmt eine separate Automatisierung. Diese Arbeitsteilung ist Absicht: sonst würden sich beide gegenseitig überschreiben.

Beispiel für die zugehörige Automatisierung (verkürzt):

```
alias: "Sicherheit: BRANDALARM"
mode: parallel
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      - binary_sensor.hitzemelder_kueche
    to: "on"
actions:
  # 1. Boolean setzen -> der Blueprint zieht sich sofort zurueck
  - action: input_boolean.turn_on
    target:
      entity_id: input_boolean.brandalarm_status

  # 2. Kritische Push-Nachricht
  - action: notify.handys
    data:
      title: "BRANDALARM!"
      message: "Ausgeloest von: {{ trigger.to_state.name }}"
      data:
        priority: high
        channel: alarm_stream

  # 3. Alle Rollos als Fluchtweg oeffnen
  - action: cover.set_cover_position
    target:
      entity_id:
        - cover.rollo_wohnzimmer
        - cover.rollo_schlafzimmer
    data:
      position: 100

  # 4. Fluchtwegbeleuchtung
  - action: light.turn_on
    target:
      entity_id: light.flur
    data:
      brightness_pct: 100
```

Der Boolean wird nicht von selbst zurückgesetzt. Solange `input_boolean.brandalarm_status` auf `on` steht, sind alle Rolladen-Automatisierungen stillgelegt – auch nach einem harmlosen Fehlalarm durch Kochdunst oder Staub. Die Automatisierungen stehen dabei weiter auf `on` und wirken völlig gesund; sie steigen nur bei jedem Durchlauf sofort in Zweig 0 aus. Der Fehler ist von außen praktisch unsichtbar. Die beiden folgenden Bausteine lösen das.

10.1 Quittierter Reset per Push-Nachricht

Ein automatischer Reset wäre gefährlich: nach einem echten Ereignis will man nicht, dass die Rollos von allein wieder zufahren. Deshalb die Lösung mit Bestätigung durch einen Menschen – zwei zusätzliche Automatisierungen.

Automatisierung A – Reset anbieten

Sobald alle Melder seit 10 Minuten wieder ruhig sind und der Boolean noch aktiv ist, kommt eine Push-Nachricht mit Aktions-Buttons:

```
alias: "Sicherheit: BRANDALARM Reset anbieten"
mode: single
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      # ... alle weiteren Melder
    to: "off"
    for:
      minutes: 10
  - trigger: state
    entity_id: input_boolean.brandalarm_status
    to: "on"
    for:
      minutes: 10
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
  - condition: state
    entity_id:
      - binary_sensor.rauchmelder_wohnzimmer
      - binary_sensor.rauchmelder_flur
      # ... alle weiteren Melder
    state: "off"
actions:
  - action: notify.handys
    data:
      title: "Brandalarm: Melder wieder ruhig"
      message: >-
        Alle Melder sind seit 10 Minuten zurueckgesetzt. Die Rolladensteuerung
        ist weiterhin STILLGELEGT. Erst bestaetigen, wenn die Lage geklaert ist.
      data:
        ttl: 0
        priority: high
        actions:
          - action: BRANDALARM_RESET
            title: "Alarm zurueecksetzen"
          - action: BRANDALARM_IGNORE
            title: "Noch nicht"
```

Automatisierung B – Reset bestätigt

Reagiert auf den Druck des Buttons in der Benachrichtigung:

```
alias: "Sicherheit: BRANDALARM Reset bestaetigt"
mode: single
triggers:
  - trigger: event
    event_type: mobile_app_notification_action
    event_data:
      action: BRANDALARM_RESET
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
actions:
  - action: input_boolean.turn_off
    target:
      entity_id: input_boolean.brandalarm_status
  - action: notify.handys
    data:
      title: "Brandalarm zurueckgesetzt"
      message: >-
        Die Rolladensteuerung ist wieder aktiv. Normaler Betrieb innerhalb
        der naechsten 5 Minuten.
```

> Wird der Button nie gedrückt, passiert schlicht nichts – die Rollos bleiben oben. Es gibt bewusst keinen Zeitablauf, der die Entscheidung dem Menschen aus der Hand nimmt. Nach dem Reset fahren die Rollos nicht schlagartig los, sondern greifen beim nächsten regulären Durchlauf, also **spätestens nach 5 Minuten** (`time_pattern`).

Beim Anlegen über die HA-API meldet der Validator ggf., `BRANDALARM_RESET` sei „not found in service registry". Das ist ein Fehlalarm des Prüfers – es handelt sich um eine Notification-Action-ID, nicht um einen Dienstnamen.

10.2 Dashboard-Karte als zweites Sicherheitsnetz

Falls das Handy einmal nicht greifbar ist, sollte der Boolean auch in der UI erreichbar sein. Eine bedingte Karte zeigt ihn nur, solange der Alarm aktiv ist, und verschwindet nach dem Zurücksetzen von selbst:

```
type: conditional
conditions:
  - condition: state
    entity: input_boolean.brandalarm_status
    state: "on"
card:
  type: tile
  entity: input_boolean.brandalarm_status
  name: BRANDALARM — Rolladensteuerung gesperrt
  icon: mdi:fire-alert
  color: red
  features:
    - type: toggle
grid_options:
  columns: full
```

Sinnvolle Position: ganz oben im Hauptdashboard, direkt unter den Personen-Badges – dort fällt sie im Ernstfall sofort auf, stört im Normalbetrieb aber nicht, weil sie unsichtbar ist.

11. Fehlersuche und Debugging

Traces lesen

Einstellungen → Automatisierungen → die betroffene Automatisierung → Menü rechts oben → Traces. Dort zeigt HA für jeden Durchlauf, welcher `choose`-Zweig gewählt wurde und mit welchen Variablenwerten. Der Reiter *Changed Variables* ist am aufschlussreichsten – dort stehen `sonne_auf_f`, `hitze_alarm`, `beschattung_aktiv`, `is_manual_override` und `aktuelle_pos` im Klartext.

> **Läuft ein Durchlauf auf einen Template-Fehler, zeigt der Trace statt eines gewählten Zweigs ein `error`-Feld und kein `execution`.** Das ist der schnellste Weg, einen Fehler in einer Variablen von einem inhaltlich falschen Zweig zu unterscheiden: Bei einem Template-Fehler wird gar kein Zweig erreicht, und das Rollo bleibt vollständig stehen – auch die Tag-Logik.

Typische Symptome

Symptom	Ursache und Abhilfe
Rollo beschattet nie	Prüfen: Innentemperatur über der Winter-Sperre? Sonne im Azimut-Fenster der gewählten Himmelsrichtung? Lux-/PV-Schwelle realistisch? Wetterentität liefert einen der erkannten Zustände?
Rollo beschattet dauernd	Meist eine zu niedrig gesetzte Außenschwelle in Verbindung mit der ODER-Logik. Außenschwelle anheben.
Rollo pendelt	Zu enger Abstand der Temperaturschwellen oder zwei Positionen zu nah beieinander (Override-Erkennung greift nicht sauber).
Rollo fährt morgens nicht hoch	Kein Präsenzmelder ausgelöst und `zeit_spaetestens` noch nicht erreicht. Oder: `sonnenaufgang_woche` aktiv und die Sonne ist im Winter noch nicht auf.
Rollo ignoriert die Tag-Logik	Höchstwahrscheinlich `is_manual_override`. Rollo einmal auf eine bekannte Position fahren (z. B. ganz auf), dann greift die Automatik wieder.
Push-Nachrichten kommen nicht an	Der Notify-Dienst existiert nicht. Unter Entwicklerwerkzeuge → Aktionen prüfen, ob `notify.<name>` aufgeführt ist. Nach Anlegen einer Notify-Gruppe ist ein Neustart nötig.
Gar nichts fährt mehr	`input_boolean.brandalarm_status` prüfen – steht der auf `on`, ist der Blueprint stillgelegt. Mit der bedingten Dashboard-Karte aus Kapitel 10.2 sieht man das sofort. **Zeigen die Traces stattdessen einen `TypeError` oder einen anderen Template-Fehler, liegt ein Fehler in einer Variablen vor – siehe Hinweis oben.**
Rollo schließt sofort bei Sonnenuntergang, obwohl ein Versatz eingestellt ist	**Blueprint-Version prüfen. Vor V87 wirkte `sunset_offset` nur auf den Sonnenuntergangs-Trigger und wurde durch `mode: restart` zusätzlich ausgehebelt. Ab V87 steckt der Versatz in der Nacht-Bedingung.**

Variablen live testen

Entwicklerwerkzeuge → Vorlage. Damit lässt sich z. B. prüfen, ob die Sonne gerade auf die Fassade scheint:

```
Azimut:    {{ state_attr('sun.sun', 'azimuth') }}
Elevation: {{ state_attr('sun.sun', 'elevation') }}
Position:  {{ state_attr('cover.rollo_wohnzimmer', 'current_position') }}
```

Für den Nachtbeginn:

```
{% set offset = 15 %}
Sonnenuntergang: {{ (state_attr('sun.sun','next_setting') | as_datetime
                     - timedelta(days=1)) | as_local }}
Nachtbeginn:     {{ (state_attr('sun.sun','next_setting') | as_datetime
                     - timedelta(days=1)
                     + timedelta(minutes=offset)) | as_local }}
```

> **`state_attr('sun.sun','next_setting')` liefert einen String, kein `datetime`.** Ohne `| as_datetime` scheitert jede Rechnung damit an einem `TypeError`. Das gilt für eigene Templates genauso wie für Anpassungen am Blueprint.

> **Bedingungs-Variablen immer als eine Expression schreiben, nie als `{% if %}`-Block.** Ein Jinja-Block liefert keinen Boolean, sondern rendert einen mehrzeiligen String. Jeder nicht-leere String ist in Jinja truthy – ein Block, der das Wort `false` ausgibt, wird also als **wahr** gewertet. Home Assistant wandelt nur exakte Python-Literale um: `False` mit großem F und ohne umgebende Leerzeichen. Genau daran ist V87 gescheitert (`ist_nacht` war rund um die Uhr wahr, die Rollos fuhren mittags zu). Richtig ist eine einzelne Expression mit `and` / `or`, die einen echten Boolean zurückgibt. Zum Prüfen im Vorlagen-Editor den Rohwert in Anführungszeichen ausgeben – steht dort etwas anderes als exakt `True` oder `False`, ist es ein String.

12. Bekannte Einschränkungen

- **Nur positionsfähige Rollos.** Reine Auf/Zu-Antriebe ohne `current_position` funktionieren nicht.
- **Keine Lamellensteuerung.** Raffstores mit Lamellenwinkel werden nur über die Höhe angesteuert.
- **Ein Rollo pro Automatisierung.** Das ist Absicht (individuelle Parameter je Fenster), bedeutet aber bei vielen Fenstern entsprechend viele Automatisierungen.
- **Keine Verschattungsberechnung** durch Nachbargebäude oder Bäume. Der Azimut-Bereich ist ein grobes Modell. Wer eine Baumreihe vor dem Westfenster hat, muss den Winkelbereich oder die Lux-Schwelle entsprechend anpassen.
- **Kein Windschutz.** Für Markisen oder außenliegende Behänge mit Windrisiko wäre eine zusätzliche, übergeordnete Sperre nötig.
- **Der Brandalarm-Boolean wird vom Blueprint nicht zurückgesetzt.** Das ist Absicht – der Reset gehört in eine eigene, quittierte Automatisierung. Siehe Kapitel 10.
- **Das `time_pattern` alle 5 Minuten bedeutet: eine Änderung an den Bedingungen wirkt sich im ungünstigsten Fall erst nach 5 Minuten aus. Das betrifft auch den Beginn des Nachtmodus.**
- **Der Sonnenuntergang wird aus `next_setting` minus einem Tag abgeleitet und weicht um die Tagundnachtgleichen um bis zu ca. 6 Minuten ab. Siehe Kapitel 7.**

13. Changelog

Version	Änderungen
V88	Fix: V87 enthielt ein defektes `ist_nacht` und **darf nicht verwendet werden**. Die Bedingung war als `{% if %}` / `{% else %}`-Block geschrieben und gab dort das nackte Wort `false` aus. Ein Jinja-Block liefert aber keinen Boolean, sondern rendert einen mehrzeiligen *String* (`"\n false\n"`) – und jeder nicht-leere String ist truthy. Home Assistant wandelt nur exakte Python-Literale um (`False`, groß geschrieben, ohne umgebenden Whitespace); kleingeschriebenes `false` blieb damit ein String und wurde als **wahr** gewertet. `ist_nacht` war folglich rund um die Uhr wahr: Block 3 griff zu jeder Tageszeit und fuhr die Rollos bei hellem Tageslicht in die Lüftungs- oder Schließposition, während Block 4 (`not ist_nacht`) überhaupt nicht mehr lief. Behoben durch eine einzelne `and`-Expression, die einen echten Boolean liefert: Sonne über dem Horizont → `False`, darunter → der Zeitvergleich. Generelle Lehre für den Blueprint: Jede Variable, die als Bedingung dient, muss eine Expression sein, niemals ein `{% if %}`-Block.
V87	*(fehlerhaft, ersetzt durch V88)* Fix: Der eingestellte **Sonnenuntergangs-Versatz war faktisch wirkungslos**. Er lag als `delay` in Block 3 und wurde nur ausgeführt, wenn `trigger.id == 'sunset_event'` war. Jeder andere Trigger – Präsenzmelder, Fensterkontakt, PV, Hausmodus, Sicherheitsnetz – lief ohne Verzögerung durch, und `ist_nacht` war ab Sonnenuntergang sofort wahr. Die Rollos fuhren dadurch binnen Minuten nach Sonnenuntergang zu, obwohl z. B. 60 Minuten eingestellt waren. Zweites, tieferliegendes Problem: Auch der `sunset_event`-Pfad hat nie funktioniert, weil die Automatisierung in `mode: restart` läuft – der laufende `delay` wurde vom nächsten Trigger abgebrochen, seit V85 (`/15` → `/5`) also spätestens nach 5 Minuten, und der Neulauf erfüllte `ist_nacht` und schloss sofort. Lösung: Der Versatz steckt jetzt in der Variable `ist_nacht` statt in einem `delay`. Damit gilt er für *jeden* Trigger und ist gegen `mode: restart` immun. Das `delay`-Konstrukt in Block 3 ist ersatzlos entfallen. Ein zusätzlicher Trigger ist nicht nötig – das bestehende `/5`-Sicherheitsnetz löst Block 3 aus, sobald `ist_nacht` kippt, die maximale Ungenauigkeit beträgt also 5 Minuten. **Alle Inputs sind unverändert** (`sunset_offset` behält Key, Typ, Default und Selector), ein Re-Import aktualisiert bestehende Automatisierungen daher ohne Neukonfiguration. Hinweis zur Umsetzung: `state_attr('sun.sun','next_setting')` liefert einen **String**, kein `datetime` – die Subtraktion braucht daher ein `| as_datetime`. Fehlt es, wirft das Template einen `TypeError`, `ist_nacht` rendert nicht, und weder Block 3 (`ist_nacht`) noch Block 4 (`not ist_nacht`) wird erreicht: Das Rollo bleibt komplett stehen. Block 4 bleibt bewusst auf `not ist_nacht`: In der neuen Dämmerungsphase (Sonne unten, Versatz läuft noch) sind `ist_hell` *und* `ist_nacht` false, und 4c nutzt dieses Fenster, um Rollos hochzufahren, die noch in Beschattungsposition stehen. Manuell geschlossene Rollos bleiben über `is_manual_override` unten.
V86	Fix: Block 4c (Auffahren) prüfte `not lueftung_aktiv` und sperrte damit das Hochfahren dauerhaft, solange ein Fenster auf Kipp stand – nicht nur während der Beschattung. Das Rollo fuhr über 4a/4b korrekt in die Beschattungs- bzw. Extrem-Position, kam nach Ende der Sonnenstandsfahrt aber nicht wieder hoch. Bedingung entfernt: Die Lüftungsposition ist im gesamten Blueprint als Mindesthöhe modelliert – 1b fährt nur hoch, wenn die aktuelle Position darunter liegt, Block 3 schließt abends nur bis dorthin, und auch 4a/4b prüfen sie nicht –, nicht als Obergrenze. `sperrschutz` bleibt in 4c erhalten, und da Block 4 nur tagsüber läuft, bleibt die nächtliche Kipp-Logik in Block 3 unberührt.
V85	Fix: Der PV-Trigger war als `state` mit `for: 2min` definiert und feuerte bei einem sich ständig ändernden Watt-Sensor praktisch nie, da das `for`-Fenster bei jeder Wertänderung neu startete – die Beschattung reagierte dadurch nie direkt auf die PV-Leistung, sondern nur im Zufallstakt. Jetzt `numeric_state` (`above` / `below` `pv_schwelle`, je `for: 2min`), was korrekt entprellt. Änderung: Sicherheitsnetz (`time_pattern`) von `/15` auf `/5` Minuten verkürzt, damit die Öffnungs-Feinkante bei `pv_schwelle − 500` bzw. `lux_schwelle − 5000` binnen maximal 5 statt 15 Minuten erkannt wird. `mode: restart` und die Positions-Guards verhindern weiterhin Mehrfachfahrten.
V84	Neu: Ganz geöffnetes Fenster fährt das Rollo ganz auf, wenn kein Sonnenstands-Modus aktiv ist (auch nachts, vor der Öffnungszeit und aus manuell geschlossenem Zustand); bei aktiver Beschattung weiterhin nur bis zur Durchgangsposition. Fix: Variablenreihenfolge – `sonne_auf_f` wird jetzt vor `hitze_extrem` definiert. Neu: Workday-Sensor als Input für Feiertagserkennung. Neu: Hilfsvariablen zur Entzerrung der Helligkeitslogik.
V83	Extrem-Hitzeschutz mit separater Position und Hysterese über Innen- und Außentemperatur.
V82	Anti-Flapping: PV-Trigger mit Entprellung, `mode: restart`, 15-Minuten-Sicherheitsnetz.

Diese Dokumentation beschreibt den Blueprint »Master-Rolladensteuerung V88«. Verwendung auf eigene Verantwortung – insbesondere die Brandalarm-Kopplung sollte vor dem Produktivbetrieb einmal vollständig durchgetestet werden (Boolean von Hand setzen und prüfen, ob alle Rollos wie erwartet reagieren).

Rot hervorgehobene Passagen kennzeichnen Änderungen gegenüber der vorherigen Dokumentationsfassung (Stand V84).